

Tema 1. Introducción

Tema 1.2 Lenguajes y gramáticas. Jerarquía de Chomsky.



Lenguajes Regulares



Descripción de lenguajes

- **Gramáticas:**
 - describen la estructura de un lenguaje.
 - Genera un lenguaje.
- **Expresión regular:**
 - Describen las cadenas aceptables o pertenecientes a un lenguajes regular
 - Caracterizan un lenguaje.
- **Autómatas:**
 - Máquinas abstractas capaces de recibir, procesar y reconocer palabras.
 - Reconocen lenguajes.



Lenguajes regulares y expresiones regulares

- **Analizadores léxicos**

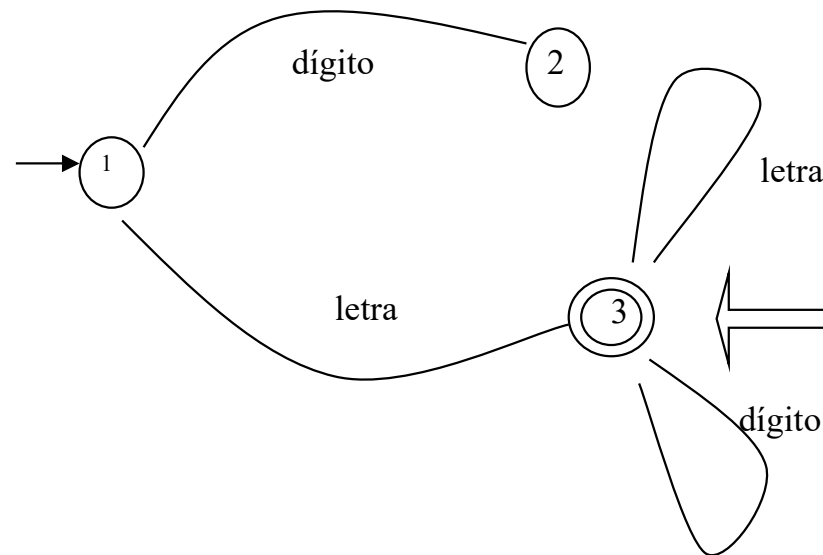
- **Problema a resolver por un compilador:**

- Detectar si una palabra del programa fuente es válida como nombre de variable

- **Solución:**

- 1) Representar de una manera concisa, no ambigua, la estructura de un nombre aceptable:

**Diagrama de Transiciones
ó
Diagrama de Estados**



Aceptaría como variable cualquier cadena de caracteres que comience por una letra seguida de letras y/o caracteres



Lenguajes regulares y expresiones regulares

Analizadores Léxicos

- Pseudocódigo:

Estado := 1;

repeat

Leer siguiente símbolo de entrada;

Case estado of

1: ***if*** *símbolo actual es una letra* ***then*** *Estado := 3*

else if *símbolo actual es un dígito* ***then*** *Estado := 2*

else *salir a la rutina de error;*

2: *Salir a la rutina de error;*

3: ***if*** *símbolo actual es una letra* ***then*** *Estado := 3*

else if *símbolo actual es un dígito* ***then*** *Estado := 3*

else *salir a la rutina de error;*

until *fin-de-cadena;*

if *Estado no es 3* ***then*** *salir a la rutina de error;*



Lenguajes regulares y expresiones regulares

Analizadores Léxicos

- El código generado desde este pseudocódigo **no** va a ser el más **eficiente** posible.
- A menudo el código resultante al partir desde el **diagrama de transición no es eficiente**
- Para construir analizadores léxicos se suele partir de una **tabla de transición**:

	letra	digito	FDC
1	3	2	Error
2	Error	Error	Error
3	3	3	Aceptar



Lenguajes regulares y expresiones regulares

Analizadores Léxicos

- Nuevo Pseudocódigo:

Estado := 1;

Repeat

Leer siguiente símbolo de entrada;

Case símbolo ***of***

Letra: Entrada := “letra”;

Dígito: Entrada:= “dígito”;

Marca de fin de cadena: Entrada := “FDC”;

En otro caso: salir a la rutina de error;

Estado := Tabla[Estado, Entrada];

If Estado = “error” ***then*** salir a la rutina de error;

Until Estado = “aceptar”



Lenguajes regulares y expresiones regulares

- Definición 2.1: Sea Σ un alfabeto. El conjunto de los **lenguajes regulares** sobre Σ se define recursivamente como:
 - a) $\emptyset$ es un lenguaje regular (LR).
 - b) $\{\lambda\}$ es un LR.
 - c) $\forall a \in \Sigma, \{a\}$ es un LR.
 - d) si A y B son LR $\Rightarrow A \cup B, A \cdot B$ y A^* son LR.
 - e) Ningún otro lenguaje sobre Σ es LR.



Lenguajes regulares y expresiones regulares

- Ejemplo: Dado $\Sigma = \{a, b\}$, las siguientes afirmaciones son ciertas:
 - $\emptyset$ y $\{\lambda\}$ son lenguajes regulares
 - $\{a\}$ y $\{b\}$ son LR
 - $\{a,b\}$ es un LR
 - $\{a, ab, b\}$ es un LR
 - $\{a^i / i \geq 0\}$ es LR
 - $\{a^i b^j / i \geq 0 \wedge j \geq 0\}$ es LR
 - $\{(ab)^i / i \geq 0\}$ es LR



Lenguajes regulares y expresiones regulares

- Definición 2.2: Expresión Regular (ER) (Kleene, 1.956) Dado un alfabeto Σ , se cumple que:
 - a) El símbolo $\emptyset$ es una expresión regular.
 - b) El símbolo λ es una expresión regular.
 - c) Cualquier símbolo $a \in \Sigma$ es una expresión regular.
 - d) Si α y β son expresiones regulares, $\alpha + \beta$ y $\alpha \cdot \beta$ también son expresiones regulares.
 - e) Si α es una expresión regular, α^* también es una expresión regular, α^* se define así:

$$\alpha^* = \sum_{i=0}^{\infty} \alpha^i$$

- f) Sólo son expresiones regulares las que se pueden obtener aplicando las reglas anteriores un número finito de veces sobre los símbolos de Σ , la palabra vacía λ , y el conjunto vacío $\emptyset$.



Lenguajes regulares y expresiones regulares

- Prioridad de las operaciones
 - 1) $*$ (cierre)
 - 2) $\cdot$ (concatenación)
 - 3) $+$ (unión)
- La prioridad se puede modificar mediante el uso de paréntesis “(“, ”)”



Lenguajes regulares y expresiones regulares

- Lenguaje $L(\alpha)$ asociado a una expresión regular α :
 1. Si $\alpha = \emptyset$, $L(\alpha) = \emptyset$
 2. Si $\alpha = \lambda$, $L(\alpha) = \{\lambda\}$
 3. Si $\alpha = a$, $a \in \Sigma$, $L(\alpha) = \{a\}$
 4. Si α y β son expresiones regulares, entonces
$$L(\alpha + \beta) = L(\alpha) \cup L(\beta)$$
 5. Si α y β son expresiones regulares, entonces
$$L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta)$$
 6. Si α es una expresión regular, entonces
$$L(\alpha^*) = L(\alpha)^*$$



Σ	Expresión regular	Lenguaje
$\Sigma=\{a,b,c,...,z\}$	$(a+b+c+...+z)^*$	Σ^*
$\Sigma=\{0,1\}$	0^*10^*	Cadenas binarias que contienen un solo 1
	$01+000$	$\{01, 000\}$
	$(0+1)^*00(0+1)^*$	Cadenas binarias que contienen al menos un par de 0 consecutivos
$\Sigma=\{a,b,c\}$	$a(a+b+c)^*$	Cadenas formadas por a , b o c que comienzan por a
	$a+bc+bba$	$\{a,bc,b^2a\}$
$\Sigma=\{a,b\}$	$a^*(a+b)$	$\{a,aa,aaa,a...a,b,ab,aab,aaab,a...ab\}$
	$(a+b)^*(a+bb)$	Cadenas formadas por a o b que terminan en a o en bb
	$(aa)^*(bb)^*b$	Cadenas formadas por un número par de símbolos a (incluso 0) seguidos por un número impar de símbolos b (al menos 1) = $\{a^{2n}b^{2m+1}: n \geq 0, m \geq 0\}$



Lenguajes regulares y expresiones regulares

- Definición 2.3: Dos expresiones regulares α y β son **equivalentes** (y lo representaremos por $\alpha = \beta$) si $L(\alpha) = L(\beta)$
- Teorema 2.1: Un lenguaje L es regular $\Leftrightarrow \exists \alpha$ expresión regular / $L(\alpha) = L$
- Demostración: Ejercicio
- Definición 2.4: llamaremos E_Σ al conjunto de todos los lenguajes regulares (es decir, los lenguajes asociados a expresiones regulares)



Lenguajes regulares y expresiones regulares

- Propiedades de las expresiones regulares

1. + es asociativa: $\alpha + (\beta + \delta) = (\alpha + \beta) + \delta$

2. + es conmutativa: $\alpha + \beta = \beta + \alpha$

3. • es asociativa: $\alpha \cdot (\beta \cdot \delta) = (\alpha \cdot \beta) \cdot \delta$

4. • es distributiva con respecto a +

$$\alpha \cdot (\beta + \delta) = (\alpha \cdot \beta) + (\alpha \cdot \delta)$$

$$(\beta + \delta) \cdot \alpha = \beta \cdot \alpha + \delta \cdot \alpha$$

5. • tiene elemento neutro (λ):

$$\alpha \cdot \lambda = \lambda \cdot \alpha = \alpha$$

6. + tiene elemento neutro ($\emptyset$):

$$\alpha + \emptyset = \emptyset + \alpha = \alpha$$



Lenguajes regulares y expresiones regulares

$$7. \quad \lambda^* = \lambda$$

$$8. \quad \emptyset \cdot \alpha = \alpha \cdot \emptyset = \emptyset$$

$$9. \quad \emptyset^* = \lambda$$

$$10. \quad \alpha^* \cdot \alpha^* = \alpha^*$$

$$11. \quad \alpha^* \cdot \alpha = \alpha \cdot \alpha^*$$

$$12. \quad (\alpha^*)^* = \alpha^*$$

Ejercicio: Demostrarlo



Lenguajes regulares y expresiones regulares

- 13. $\alpha^* = \lambda + \alpha + \alpha^2 + \dots + \alpha^n + \alpha^{n+1} \cdot \alpha^*$
- 14. $\alpha^* = \lambda + \alpha \cdot \alpha^*$ (caso particular para $n=0$)
- 15. $\alpha^* = (\lambda + \alpha)^{n-1} + \alpha^n \cdot \alpha^*$
- 16. $(\alpha^* + \beta^*)^* = (\alpha^* \cdot \beta^*)^* = (\alpha + \beta)^*$
- 17. $(\alpha \cdot \beta)^* \cdot \alpha = \alpha \cdot (\beta \cdot \alpha)^*$
- 18. $(\alpha^* \cdot \beta)^* \cdot \alpha^* = (\alpha + \beta)^*$
- 19. $(\alpha^* \cdot \beta)^* = (\alpha + \beta)^* \cdot \beta + \lambda$
 $(\alpha^* \cdot \beta)^* = \alpha \cdot (\alpha + \beta)^* + \lambda$



Lenguajes regulares y expresiones regulares

20. Reglas de inferencia

- Sean R, S, T expresiones regulares. Se verifica que

$$R = S^* \cdot T \Rightarrow R = S \cdot R + T$$

- Si $\lambda \notin S$, entonces

$$R = S \cdot R + T \Rightarrow R = S^* \cdot T$$



Repaso y preguntas con intención...

1. ¿Cuántas palabras tiene el lenguaje asociado a la expresión regular $(a+b)^*$?
2. ¿Qué expresión regular tiene asociado el lenguaje de todas las palabras sobre el alfabeto $\{a,b,c\}$ que solo tienen una letra c ?
3. ¿Cual es la expresión regular sobre el alfabeto $\{a, b\}$ que tiene como lenguaje asociado el lenguaje formado por las palabras que tienen el mismo número de letras “a” que de letras “b” ?
(¡cuidado! Parece más fácil de lo que es...)

Análisis léxico

- ¿Cuál fue la necesidad de los lenguajes formales?
- Definiciones básicas.
- Gramáticas formales.
- Derivaciones y árboles de derivación.
- Ambigüedad.
- Eliminación de la ambigüedad.



¿Cuál fue la necesidad de los lenguajes formales?

- Allá por 1954, se inició el desarrollo del **lenguaje de programación FORTRAN** que, aunque fue un verdadero compilador, adolecía de una **gran complejidad**.
- **Noam Chomsky** empezó con el estudio del **lenguaje natural**, con la idea de estructurarlo, y consiguió una revolución en este campo.

Gramáticas Tipo 0 **Sin restricciones.** Estas gramáticas tienen que tener en su parte izquierda al menos un símbolo no terminal.

Gramáticas Tipo 1 **Dependientes del contexto.** Se las denomina **dependientes del contexto** porque hay que tener en cuenta los símbolos que vienen antes y después del que queremos sustituir (su contexto).

Gramática Tipo 2 **Independientes del contexto.** Generan lenguajes independientes del contexto y se caracterizan porque en la parte izquierda de una producción solo pueden tener un símbolo no terminal.

Gramática Tipo 3 **Expresiones regulares.** Estas son las gramáticas más restrictivas y generan lenguajes regulares. En su parte izquierda tienen solo un **no terminal** y en su parte derecha tienen solo un **terminal**.



Gramáticas Formales

- **Una gramática formal** es una descripción de la estructura de las sentencias que forman un lenguaje, entendiendo por lenguaje, en este contexto, **un lenguaje de programación**.
- La forma en la que se describe una gramática (G) es mediante cuatro términos:
 - El alfabeto de los símbolos terminales (Σ_T),
 - El alfabeto de los símbolos no terminales (Σ_N),
 - El axioma o símbolo inicial de la gramática (S)
 - Un conjunto de producciones (P).
- Se denota de la siguiente forma:

$$G = \{\Sigma_T, \Sigma_N, S, P\}$$



Jerarquía de Chomsky

Gramáticas Tipo 0

Sin restricciones. Estas gramáticas tienen que tener en su parte izquierda al menos un símbolo no terminal.

Gramáticas Tipo 1

Dependientes del contexto. $xAy \rightarrow xBy$

Gramática Tipo 2

Independientes del contexto. $A \rightarrow X$

Gramática Tipo 3

Expresiones regulares. $A \rightarrow aB \mid A \rightarrow Ba .$



Gramáticas Formales

- **Símbolos terminales (Σ_T):** Representa al conjunto de palabras reservadas de un lenguaje de programación. Esto incluye los caracteres especiales y los operadores tanto aritméticos como lógicos. Ejemplo: En el lenguaje de programación C:
 - $\Sigma_T = \{\text{main}, \{, \}, \#, \text{include}, \text{if}, \text{then}, \text{define}, \text{int}, \text{float}, \text{char}, \text{double}, ;, +, -, *, >, >=, |, \dots\}$
 - Se denotan con letras minúsculas: Ej: **int, float, if**
- **Símbolos no terminales (Σ_N):** Representa el conjunto de variables que utilizamos en las producciones de las gramáticas como símbolos de transición. Esto incluye el símbolo inicial de la gramática (S) y las variables que utilicemos.
 - Se denota con letras mayúsculas.



Gramáticas Formales

- **Símbolo inicial (S):** Es un símbolo no terminal a partir del cual se obtienen todas las palabras del lenguaje y será el primer símbolo de la gramática G . También se le denomina **el *axioma*** de la gramática.
- **Conjunto de producciones (P):** Es un conjunto de reglas que indica como se obtienen las palabras del lenguaje definido por G . Establece las relaciones entre los símbolos terminales y los no terminales.
- Para definir las se utiliza la **Forma de Backus-Naur (BNF)**. Su origen está en la especificación del lenguaje de programación ALGOL60, a mediados de los 60. La BNF es un metalenguaje, es decir, un lenguaje con el que se describen otros lenguajes y se describe de acuerdo con la siguiente tabla:



Gramáticas Formales

- Conjunto de producciones (P):

Símbolo	Significado
$\rightarrow$ $::=$	“Se define como”. Se utilizan los dos símbolos indistintamente
	alternativa
[a]	Una o ninguna ocurrencia de a. Expresan opcionalidad
{a}	Número arbitrario de ocurrencias de a (cero o más repeticiones)

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow - E$

$E \rightarrow (E)$

$E \rightarrow \text{id}$



Derivaciones y Árboles de Derivación

- Se trata de ver cómo, a partir del símbolo inicial de la gramática se puede derivar una secuencia de símbolos utilizando las producciones de esa gramática y esto se hace mediante derivaciones.
- Estas derivaciones se representan graficamente mediante un **árbol de derivación** o también llamado **Árbol de Análisis Sintáctico**.
 - ¿Como lo vemos? (Ejercicio)

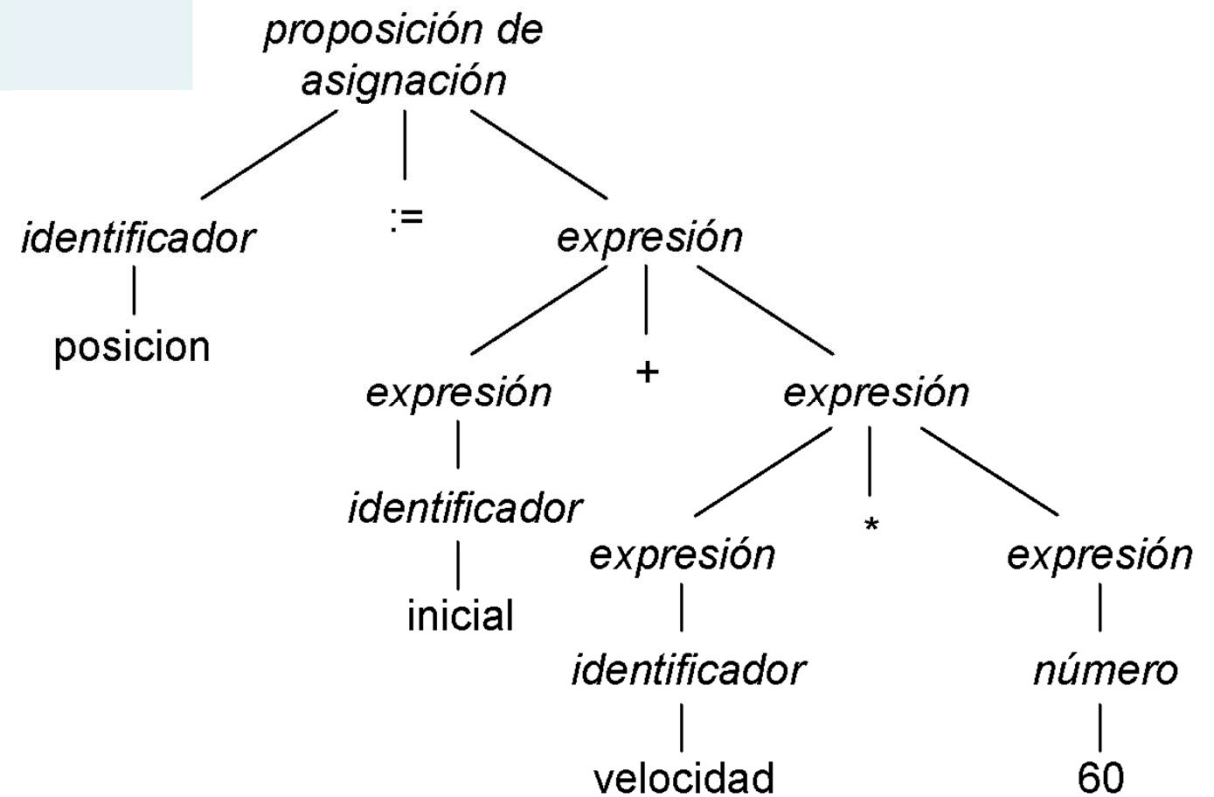
Gramática	Sentencia
$E \rightarrow E + E$ $E \rightarrow E * E$ $E \rightarrow - E$ $E \rightarrow (E)$ $E \rightarrow id$	(id * id)



Derivaciones y Árboles de Derivación

Gramática	Sentencia
$E \rightarrow E + E$ $E \rightarrow E * E$ $E \rightarrow - E$ $E \rightarrow (E)$ $E \rightarrow id$	(id * id)

“Pista” (de la semana pasada):



Ambigüedad

- **La ambigüedad** se produce cuando hay más de una manera de reconocer una palabra o sentencia a partir del símbolo inicial de la gramática.
- La ambigüedad se puede dar a varios niveles: sentencia, gramática o lenguaje.
 - Una sentencia es ambigua si existe más de una derivación para reconocerla en una gramática.
 - Una gramática es ambigua si existe en su lenguaje una sentencia ambigua, es decir, es posible derivar una sentencia por más de una derivación.
 - Un lenguaje es ambiguo si existe una gramática ambigua que lo genera. Si todas las gramáticas que lo generan son ambiguas, entonces se denomina al lenguaje **inherentemente ambiguo**.

NOTA: El problema de resolver si un lenguaje ambiguo cualquiera es inherentemente ambiguo es un **problema irresoluble**.



Ambigüedad

- **Recursividad por la izquierda:** Se produce este tipo de recursividad cuando el primer símbolo no terminal aparece tanto en la parte derecha como en la izquierda de una producción.
 - Ejemplo: $A \rightarrow A\alpha$, donde α representa cualquier combinación de terminales y no terminales (no tiene porque ser un solo símbolo). Veremos más adelante como se elimina esta recursividad.
- **Recursividad por la derecha:** Si el símbolo no terminal que aparece el último en la parte derecha de la producción es igual al de la parte izquierda.
 - Ejemplo: $A \rightarrow \alpha A$
- **Factorizar por la izquierda:** Esta solución se aplica cuando hay varias alternativas en una sentencia que comienzan igual . Ejemplo: $A \rightarrow B\alpha \mid Bx$, tenemos que eliminar esta ambigüedad reescribiendo las producciones



Eliminar la ambigüedad

- **Eliminar la recursividad a izquierdas:** Partimos de la siguiente producción $A \rightarrow A\alpha \mid \beta$, donde tanto α como β representan conjuntos de terminales y no terminales. La regla a aplicar es la siguiente:

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A \mid \lambda \end{aligned}$$

- Ejemplo: $E \rightarrow E + T \mid T$, donde $\alpha = + T$ y $\beta = T$. Por tanto el resultado sería:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + TE \mid \lambda \end{aligned}$$



Eliminar la ambigüedad

- **Factorizar por la izquierda:** Puesto que parte de alternativas de una producción que comienzan igual, tenemos que determinar la parte común más larga entre estas alternativas y a partir de ahí se sustituye por un no terminal que actuará de discriminante. Si tenemos $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \theta$, donde θ representa otras alternativas que no comienzan con α , se hace lo siguiente:

$$A \rightarrow \alpha A' \mid \theta$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- Ejemplo: $E \rightarrow S d E \mid S$ donde la parte común, α , es igual a S , por tanto aplicando la regla anterior:

$$E \rightarrow S A'$$

$$A' \rightarrow d E \mid \lambda$$

